Lecture 2

- Regular Language and Regular Expression.
- Exercises

Language Concatenation

- If $w \in \Sigma^*$ and $x \in \Sigma^*$, then wx is the ***concatenation*** of w and x.
- If L_1 and L_2 are languages over Σ, the ***concatenation*** of L_1 and L_2 is the language L_1L_2 defined as

$$L_1L_2 = \{ wx \mid w \in L_1 \text{ and } x \in L_2 \}$$

- Example: if $L_1 = \{$ a, ba, bb $\}$ and $L_2 = \{$ aa, bb $\}$, then

$$L_1L_2 = \{ \text{aaa}, \text{abb}, \text{baaa}, \text{babb}, \text{bbaa}, \text{bbbb} \}$$

Lots and Lots of Concatenation

- Consider the language L = { aa, b }
- LL is the set of strings formed by concatenating pairs of strings in L.

  { aaaa, aab, baa, bb }

- LLL is the set of strings formed by concatenating triples of strings in L.

  { aaaaaa, aaaab, aabaa, aabb, baaaa, baab, bbaa, bbb}

- LLLL is the set of strings formed by concatenating quadruples of strings in L.

  { aaaaaaaa, aaaaaab, aaaabaa, aaaabb, aabaaaa, aabaab, aabbaa, aabbb, baaaaaa, baaaab, baabaa, baabb, bbaaaa, bbaab, bbbaa, bbbb}

The Kleene Closure

- An important operation on languages is the ***Kleene Closure***, which is defined as

$$L^* = \{ w \in \Sigma^* \mid \exists n \in \mathbb{N}.\ w \in L^n \}$$

- Mathematically:

$$w \in L^* \quad \textbf{iff} \quad \exists n \in \mathbb{N}.\ w \in L^n$$

- Intuitively, all possible ways of concatenating zero or more strings in L together, possibly with repetition.

The Kleene Closure

If L = { a, bb }, then L^* = {

ε,

a, bb,

aa, abb, bba, bbbb,

aaa, aabb, abba, abbbb, bbaa, bbabb, bbbba, bbbbbb,

...

}

Closure Properties

- ***Theorem:*** If L_1 and L_2 are regular languages over an alphabet Σ, then so are the following languages:
  - $\overline{L_1}$
  - $L_1 \cup L_2$
  - $L_1 \cap L_2$
  - $L_1 L_2$
  - L_1^*
- These properties are called ***closure properties of the regular languages***.

Nothing else is a RL.

Examples RL

- {ε} is regular:
  - the empty language ϕ is regular, ϕ^*= {ε} is regular.
- Σ={0,1} → {011} is regular.
  - {0} and {1} are regular, {011}={0}{1}{1} is regular

Every language containing only one string is regular.

{011} and {100} are regular,
→ {011, 100} = {011}U{100} is regular.

Every finite language is regular.
Every infinite RL can be obtained with Kleene closure

(1) Kleene closure has the highest preference
(2) Concatenation (and)
(3) Union. (or)

1	*
2	○
3	∪

Op Preference

Ex. ({0}*U{0}{1}{1}*) {0}{0}{1}* ≡ . (0* | 011*) 001*

The previous RE contains too many parentheses ..... use simpler RE (condensed notation)

RE condensed notation

1) ϕ is a RE of the empty language.
2) **ε** is a RE of {ε}.
3) For any unique symbol a, **a** is a RE
4) If **rA** and **rB** are REs of languages A and B, then:

   rA | rB ; **rA+rB** : RE of A U B,

   rA rB ; **rA . rB** : RE of AB

   rA* : RE of A*.

011 : RE of {0}{1}{1}.
0+1 : RE of {0,1}.
(0+1)* : RE of {0,1}*.
(0+1) $^+$: RE of $\{0, 1\}^+$.

1? : Denotes 0 or 1 occurrence

Designing Regular Expressions

- Let $\Sigma = \{$a, b$\}$.
- Let $L = \{ w \in \Sigma^* \mid w$ contains aa as a substring $\}$.

(a | b)*aa(a | b)*

bbabbbaabab
aaaa
bbbbbabbbbaabbbbb

Designing Regular Expressions

- Let $\Sigma = \{$a, b$\}$.
- Let $L = \{\ w \in \Sigma^* \mid w$ contains at most one a $\}$.

b*a?b*

bbbbabbb
bbbbbb
abbb
a

Show that the language of all binary strings starting with 01 is regular.

Proof. A word in this language is in form **01** $x_1 \cdots x_n$ where $x_1 \cdots x_n \in \{0,1\}^*$.
Therefore, the language can be written as:

$\{01\}\ \{0,1\}^* = (01)(0|1)^*$ which is regular (bcse can be expressed as a RE).

Show that the language of all binary strings ending with 01 is regular.

Proof. A word in this language is in form $x_1 \cdots x_n$ **01** where $x_1 \cdots x_n \in \{0,1\}^*$.
Therefore, the language can be written as:

$\{0,1\}^* \{01\} = (0|1)^* \ (01)$ which is regular (bcse can be expressed as a RE).

Show that the language of all binary strings <u>having substring 01 is regular.</u>

Proof. A word in this language has the form:

$$x_1 \cdots x_n \mathbf{01} y_1 \cdots y_m \qquad \text{where } x_1 \cdots x_n, y_1 \cdots y_m \in \{0,1\}^*.$$

Therefore, the language can be written as:

$$\{0,1\}^* \{01\} \{0,1\}^* \equiv$$

$$(\{0\} \cup \{1\})^* \mathbf{(\{0\}\{1\})} (\{0\} \cup \{1\})^*$$

$$\equiv$$

$$(0|1)^* \ \mathbf{(01)} \ (0|1)^*$$

which is regular.

Exercise (without proof)

Give valid REs for the below languages description over $\Sigma = \{0,1\}$ (i.e. Binary alphabet)

1. starting with 01: **01(0|1)***
2. ending at 01: **(0|1)*01**
3. having substring 01: **(0|1)*01(0|1)***

How many REs can a language have?

- REs are a way to describe a (RL), but the description itself is not unique.
- A RL can be represented by different equivalent REs

if A is regular, so is A^{rev}.

- For a string $x=x_1x_2\ldots x_n$, $x^r=x_n\ldots x_2x_1$.
- For a language A, $A^r = \{x^r \mid x \in A\}$.

Show that if A is regular, so is A^r.

Proof.

1) $\phi^r = \phi$ is regular.
2) For any symbol a, $\{a\}^r = \{a\}$ is regular.
3) Suppose that for RLs A and B, A^r and B^r are regular.

Then

$(A \cup B)^r = A^r \cup B^r$ is regular,

$(AB)^r = B^r A^r$ is regular.

$(A^*)^r = (A^r)^*$ is regular.

Shorthand Summary

- R^n is shorthand for RR … R (n times).
  - Edge case: define $R^0 = \varepsilon$.
- Σ is shorthand for "any character in Σ."
- R? is shorthand for (R ∪ ε), meaning "zero or one copies of R."
- R^+ is shorthand for RR*, meaning "one or more copies of R."

Exercises : design **REs from Language Descriptions ?**

1. **Language of strings over Σ={a, b} that contain exactly two a's.**
2. **Language of strings over Σ={0,1} that start and end with the same symbol.**
3. **language of strings over Σ={x, y} that never contain "yy".**
4. **Language of strings over {a, b} that contain at least one "ab".**
5. **language of strings over {0,1} where every "1" is immediately followed by at least one "0".**

1- the language of strings over Σ={a, b} that contain exactly two a's.

b*ab*a b*

2-Language of strings over Σ={0,1} that start and end with the same symbol.

0(0|1)*0 | 1(0|1)*1

3-language of strings over Σ={x, y} that never contain "yy".

Extra symbol ? Denotes 0 or 1 occurrence

(x | yx)*y?

4-Language of strings over {a, b} that contain at least one "ab".

(a|b)*ab(a|b)*

5- language of strings over {0,1} where every "1" is immediately followed by at least one "0".

(0 | 10)+

- No isolated "1"
- Every "1" is followed by at least one "0"

Exercises (ctd)

6- language of words **over Σ={0,1}** not containing subword 11 and not ending with 1.

7- language of strings not containing substring 11 and ending with 0

8-over {0,1} where every "1" is followed by exactly two "0"s

9- over {x, y, z} that start and end with the same symbol

10- over {0,1} that contain at least two "1"s

11- over {0,1} that contain at least non-adjacents two "1"s

6- language of words **over Σ={0,1}** not containing subword 11 and not ending with 1.

(0|10).*

A string generated by this expr is a sequence of blocks, where each block is either a single **0** or a sequence **10**.

- **Does not contain 11:**
  - The only way a 1 appears in the string is as part of the **10** block.
  - Since every 1 is immediately followed by a 0 (the next symbol in the 10 block), it is impossible to have 11 as a substring.
  - *The language satisfies the "not containing 11" condition*.

6- language of words not containing subword 11 and not ending with 1.

(0|10).*

- **Not ending with 1:**
  - The string is formed by concatenating 0's and 10's.
  - If the last block is a **0**, the string ends with 0.
  - If the last block is a **10**, the string ends with 0.
  - Since all valid blocks end in 0, any string formed by concatenations of these blocks must also end in 0.

7- language of strings not containing substring 11 and ending with 0

$\{0, 10\}^* = (0 \mid 10)^*0$

Examples in the language:

- `0`
- `10`
- `010`
- `1010`
- `01010`

Not in the language:

- `11` (contains forbidden substring)
- `1` (ends in "1")
- `1011` (contains "11")

8-over {0,1} where every "1" is followed by exactly two "0"s

(0 | 100)*

This allows:

"0" is allowed anywhere

"1" must be followed by exactly two "0"s → "100"

So we build strings from "0" and "100" blocks

9- over {x, y, z} that start and end with the same symbol

x(x|y|z)*x | y(x|y|z)*y | z(x|y|z)*z

This allows word to Start and end with :

- ❑"x"
- ❑"y"
- ❑"z"

10- over {0,1} that contain at least two "1"s

at least two `1`**s** appear somewhere in the string, possibly separated by other characters.

$$(0|1)^*1(0|1)^*1(0|1)^*$$

Matching Strings

- 11
- 101
- 1100
- 001010
- 111

🚫 **Non-Matching Strings**

- 0
- 1
- 0000
- 01000

11- over {0,1} that contain at least two non-adjacents "1"s

$$(0|1)^*1(0^+)(0|1)^*1(0|1)^*$$

At least two 1s exist.
They are **separated by at least one** 0, so they are not adj.

Matching Strings

- 101
- 1001
- 01010
- 0010010

🚫 **Non-Matching Strings**

- 11 (adj)
- 1110 (first two are adj)
- 1 (only one 1)
- 0000 (no 1s)

12- over {a, b} that contain an even number of a’s and an odd number of b’s

Impossible

This language is **not regular**, so **no regular expression** can describe it.